ASSGNMENT_OS_LAB#06

RAMSHA AROOJ_110763

Linux Commands

System Q#1: Simulation FCFS Scheduling algorithm with the help of code.

At first open terminal and type gedit question1. c.

```
ramsha@myVM:~$ gedit question1.c
```

An editor will display, write code on it and then press Ctrl+S or click on save option.

Code:

```c
#include<stdio.h>
#include<stdlib.h>
struct process{
char pid[10];
int arrival;
int burst;
int waiting;
int turnaround;
};
int main(){
FILE *fp;
fp=fopen("input.txt","r");
if(fp==NULL){
printf("Error opening file!\n");
return 1;
}
struct process p[50];
int n=0;
while(fscanf(fp,"%s %d %d",p[n].pid,&p[n].arrival,&p[n].burst)!=EOF){
n++;
}
fclose(fp);
int time=0;
printf("\nprocess\tarrival\tburst\twaiting\tturnaround\n:
for(int i=0;i<n;i++){
if(time<p[i].arrival){
```

```c
if(time<p[i].arrival){
time=p[i].arrival;
}
p[i].waiting=time-p[i].arrival;
time+=p[i].burst;
p[i].turnaround=p[i].waiting+p[i].burst;
printf("%s\t%d\t%d\t%d\t%d\n",p[i].pid,p[i].arrival,p[i].burst,p[i].waiting,p[i].turnaround);
}
return 0;
}
```

Output:

```
ramsha@myVM:~$ gcc question1.c -o question1
ramsha@myVM:~$ ./question1

process arrival burst   waiting turnaround
p1      0       5       0       5
p2      1       3       4       7
p3      2       8       6       14
```

Q#2: Simulation Banker's Algorithm for deadlock avoidance.

Type gedit question2.c write code and compile it.

```
ramsha@myVM:~$ gedit question2.c
ramsha@myVM:~$
```

Code:

```c
#include<stdio.h>
int main(){
int n,m;
printf("Enter the number of processes: ");
scanf("%d",&n);
printf("Enter number of resources: ");
scanf("%d",&m);
int alloc[n][m], max[n][m], need[n][m], avail[m];
int finish[n], safeseq[n];
printf("\nEnter Allocation matrix:\n");
for(int i=0;i<n;i++)
for(int j=0;j<m;j++)
scanf("%d",&alloc[i][j]);
printf("\nEnter maximum matrix:\n");
for(int i=0;i<n;i++)
for(int j=0;j<m;j++)
scanf("%d",&max[i][j]);
printf("\nEnter available resources:\n");
for(int i=0;i<m;i++)
scanf("%d",&avail[i]);
for(int i=0;i<n;i++){
finish[i]=0;
for(int j=0;j<m;j++)
need[i][j]=max[i][j]-alloc[i][j];
}
printf("\nNeed matrix:");
for(int i=0;i<n;i++){
for(int j=0;j<m;j++)
```

```
for(int j=0;j<m;j++)
printf("%d", need[i][j]);
printf("\n");
}
int count=0;
while(count<n){
int found=0;
for(int p=0;p<n;p++){
if(!finish[p]){
int ok=1;
for(int j=0;j<m;j++)
if (need[p][j]>avail[j])
ok=0;
if(ok){
for(int j=0;j<m;j++)
avail[j]+=alloc[p][j];
safeseq[count++]=p;
finish[p]=1;
found=1;
    }
  }
}
if(!found){
printf("\nSystem is not in safe state!\n");
return 0;
 }
}
```

```c
printf("\nSystem is in safe state!\nsafe sequence:");
for(int i=0;i<n;i++){
printf("p%d", safeseq[i]);
printf("\n");
return 0;
  }
}
```

Output:

```
ramsha@myVM:~$ gedit question2.c
ramsha@myVM:~$ gcc question2.c -o question2
ramsha@myVM:~$ ./question2
Enter the number of processes: 5
Enter number of resources: 3

Enter Allocation matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter maximum matrix:
7 5 3
3 2 2
9 0 2
4 2 2
5 3 3

Enter available resources:
3 3 2
```

```
Need matrix:743
122
600
211
531

System is in safe state!
safe sequence:p1
```